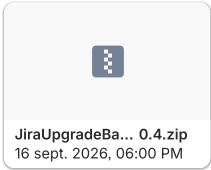


Guide_Deploiement_Nouvelle_Machine_JiraUpgradeBaselineToolkit_v0.3

Guide de déploiement — JiraUpgradeBaselineToolkit

Installation et premier lancement sur une nouvelle machine Windows à partir du dossier versionné

v0.3 — Mise à jour du 16 septembre 2026

Produit	JiraUpgradeBaselineToolkit
Architecture	Python + ScriptRunner + PowerShell de déploiement
Cible	Jira Data Center 11.3 LTS — patch exact optionnel
Point d'entrée	.\run.ps1
Zip	

1. Objet et scénario de départ

Ce guide part du cas suivant : vous disposez du dossier JiraUpgradeBaselineToolkit provenant de Git ou d'une copie des fichiers versionnés. Les éléments ignorés par Git ne sont donc pas présents : .env, .venv, output et caches Python.

Objectif : reconstruire l'environnement local puis lancer le toolkit avec un minimum de commandes, grâce à deployment\install.ps1 et run.ps1.

2. Ce qui est fourni et ce qui est recréé

Élément	Fourni	Action
Code Python	Oui	Aucune
run.ps1	Oui	Point d'entrée utilisateur
deployment/install.ps1	Oui	Installe/reconstruit l'environnement
deployment/check.ps1	Oui	Contrôle l'installation
requirements.txt	Oui	Utilisé par install.ps1
.env.example	Oui	Modèle de configuration
.env	Non	Créé localement puis complété
.venv	Non	Créé automatiquement
output	Non	Créé automatiquement
.git	Selon le mode de copie	Non requis pour exécuter

3. Prérequis machine

- Windows 10/11 ou Windows Server avec PowerShell.
- Python 3.10 ou supérieur accessible par la commande python.
- Accès réseau HTTP/HTTPS vers Jira.
- ScriptRunner actif sur l'instance Jira.
- Custom REST Endpoint upgradeBaseline déployé et protégé.
- PAT Jira Data Center pour un compte autorisé.

```
python --version
```

Exemple du labo : Python 3.14.3 a été utilisé pendant les tests v0.3.

4. Copier le dossier du toolkit

Copier le dossier dans un emplacement local. Éviter si possible les emplacements où des politiques de synchronisation ou de sécurité bloquent l'exécution des scripts.

```
cd "C:\Tools\JiraUpgradeBaselineToolkit"  
Get-ChildItem
```

Vérifier notamment la présence de run.ps1, main.py, requirements.txt, .env.example et du dossier deployment.

5. Installer l'environnement local

La méthode recommandée v0.3 est une seule commande :

```
.\deployment\install.ps1
```

Le script doit :

- vérifier que Python est disponible ;
- créer .venv si nécessaire ;
- mettre à jour pip ;
- installer requirements.txt ;
- copier .env.example vers .env si .env n'existe pas ;
- créer output/json, output/excel et output/logs ;
- effectuer un contrôle d'import simple.

Ne pas utiliser -Force au premier test : l'option -Force supprime et reconstruit .venv. Elle est utile pour une réinstallation propre, pas pour un usage courant.

6. Si PowerShell bloque les scripts

En environnement d'entreprise, appliquer la politique de sécurité de l'organisation. Pour un environnement de développement local, les options usuelles sont :

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

# ou uniquement pour la session courante
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

7. Configurer .env

Après install.ps1, ouvrir le fichier .env créé à partir de .env.example et renseigner les valeurs propres à l'instance.

```
JIRA_URL=https://jira.exemple.local
JIRA_PAT=COLLER_LE_PAT_LOCAL

INSTANCE_NAME=AXA-JIRA01

TARGET_JIRA_FAMILY=11.3
TARGET_JIRA_VERSION=

BASELINE_ENDPOINT=/rest/scriptrunner/latest/custom/upgradeBaseline

REQUEST_TIMEOUT=30
MAX_RETRIES=4
VERIFY_SSL=true

OUTPUT_DIR=output
```

Secret : JIRA_PAT ne doit jamais être ajouté à Git. Le fichier .env doit rester local et ignoré.

8. Vérifier l'installation

```
.\deployment\check.ps1
```

Résultat attendu : présence de l'environnement Python, de .env, requirements.txt et main.py, configuration lisible, imports Python disponibles et Health Check : OK.

```
[OK] Environnement Python
[OK] Fichier .env present
[OK] requirements.txt present
[OK] main.py present
...
Imports Python : OK
[OK] Modules Python disponibles

Health Check : OK
```

Comportement corrigé en v0.3 : le health check conserve la sortie Python visible et utilise le véritable code retour ; il ne doit plus afficher OK si Python a échoué.

9. Vérifier l'endpoint ScriptRunner

Le Custom REST Endpoint doit être accessible avec le PAT et retourner une baseline JSON. La version source est récupérée automatiquement par cette collecte.

Erreur	Signification probable
401	PAT absent, invalide ou expiré.
403	Compte authentifié mais non autorisé par la restriction de groupes.

404	Endpoint absent, nom incorrect ou ScriptRunner indisponible.
Timeout	Problème réseau, proxy, firewall ou Jira indisponible.

10. Lancer le toolkit

```
.\run.ps1
```

Le menu doit proposer PRE, POST, comparaison, tests et vérification d'installation. L'utilisateur n'a pas besoin d'activer .venv manuellement.

```
1 - Collecte PRE Upgrade
2 - Collecte POST Upgrade
3 - Comparaison PRE / POST
4 - Tests Python
5 - Verification installation
0 - Quitter
```

11. Tester le déploiement

1. Exécuter .\run.ps1 -Mode check.
2. Exécuter .\run.ps1 -Mode test.
3. Exécuter .\run.ps1 -Mode pre.
4. Vérifier la création d'un JSON PRE et d'un Excel PRE.
5. Ouvrir le classeur et vérifier Summary, Baseline, Findings et Readiness.

Validation machine vierge : la chaîne install.ps1 doit encore être testée de bout en bout sur une machine réellement vierge avant de figer définitivement le packaging v0.3.

12. Premier PRE

```
.\run.ps1 -Mode pre
```

Le PRE crée les fichiers de référence. Exemple de noms :

```
output\json\AXA-JIRA01_PRE_20260916_120000.json  
output\excel\AXA-JIRA01_PRE_20260916_120000.xlsx
```

13. POST et comparaison

```
.\run.ps1 -Mode post  
.\run.ps1 -Mode compare
```

COMPARE nécessite au moins un PRE et un POST pour la même INSTANCE_NAME. run.ps1 vérifie désormais cette condition avant l'appel de main.py.

Test de validation : si PRE et POST sont tous deux 10.3.25 alors que TARGET_JIRA_FAMILY=11.3, la ligne JIRA.version doit être FAIL. Les éléments stables identiques peuvent rester PASS.

14. Mettre à jour le patch cible

Lorsque le patch exact est confirmé, modifier uniquement .env :

```
TARGET_JIRA_FAMILY=11.3  
TARGET_JIRA_VERSION=11.3.11
```

Il n'est pas nécessaire de modifier main.py ni le comparateur. Le POST sera alors vérifié par rapport à la cible exacte.

15. Déployer vers une autre instance Jira

Le code reste identique. Les principales valeurs à changer sont JIRA_URL, JIRA_PAT et INSTANCE_NAME. La version source est détectée automatiquement.

1. Copier le dossier versionné.
2. Exécuter `.\deployment\install.ps1`.
3. Créer/configurer le `.env` local de l'instance.
4. Exécuter `.\deployment\check.ps1`.
5. Lancer `.\run.ps1 -Mode pre`.

16. Git local

Git n'est pas nécessaire pour exécuter le toolkit, mais il peut être activé localement pour versionner le code et les scripts.

```
git status
git log --oneline
git remote -v
```

Le projet actuel utilise un dépôt Git local. `.env`, `.venv` et `output` doivent rester ignorés.

17. Mise à jour d'une machine déjà installée

Après réception d'une nouvelle version du code, remplacer/mettre à jour les fichiers versionnés sans écraser `.env` et `output`. Si `requirements.txt` a changé, relancer `install.ps1` afin de remettre les dépendances à niveau.

```
.\deployment\install.ps1
```

Conservation : `install.ps1` ne doit pas remplacer un `.env` existant. Les secrets et paramètres locaux sont conservés.

18. Dépannage

Symptôme	Action
python introuvable	Installer Python et vérifier le PATH.
.venv absent	Relancer .\deployment\install.ps1.
.env absent	Relancer install.ps1 ou copier .env.example vers .env.
Health Check KO	Lire le contrôle KO, corriger puis relancer check.ps1.
PRE/POST absent	Lancer la collecte correspondante.
COMPARE refuse de démarrer	Vérifier output/json et INSTANCE_NAME.
401/403/404	Vérifier PAT, groupe ScriptRunner et endpoint.
Erreur SSL	Corriger le certificat ; éviter VERIFY_SSL=false en production.

19. Checklist de mise en service

Contrôle	OK
Python disponible	☐
Dossier toolkit complet	☐
install.ps1 exécuté	☐
.venv présent	☐
.env créé et renseigné	☐
PAT non versionné	☐

Health Check OK	☐
Tests pytest OK	☐
Endpoint ScriptRunner accessible	☐
PRE généré	☐
Excel PRE lisible	☐

20. Procédure condensée

```
cd "C:\Tools\JiraUpgradeBaselineToolkit"
```

```
.\deployment\install.ps1
```

```
# Editer .env
```

```
.\deployment\check.ps1
```

```
.\run.ps1 -Mode test
```

```
.\run.ps1 -Mode pre
```

Après l'upgrade :

```
.\run.ps1 -Mode post
```

```
.\run.ps1 -Mode compare
```

21. Résumé

Le modèle de déploiement v0.3 est volontairement simple : le dossier versionné apporte le code et les scripts ; install.ps1 reconstruit l'état local ; .env contient la configuration/secrets ; check.ps1 valide la machine ; run.ps1 devient l'interface d'exploitation.